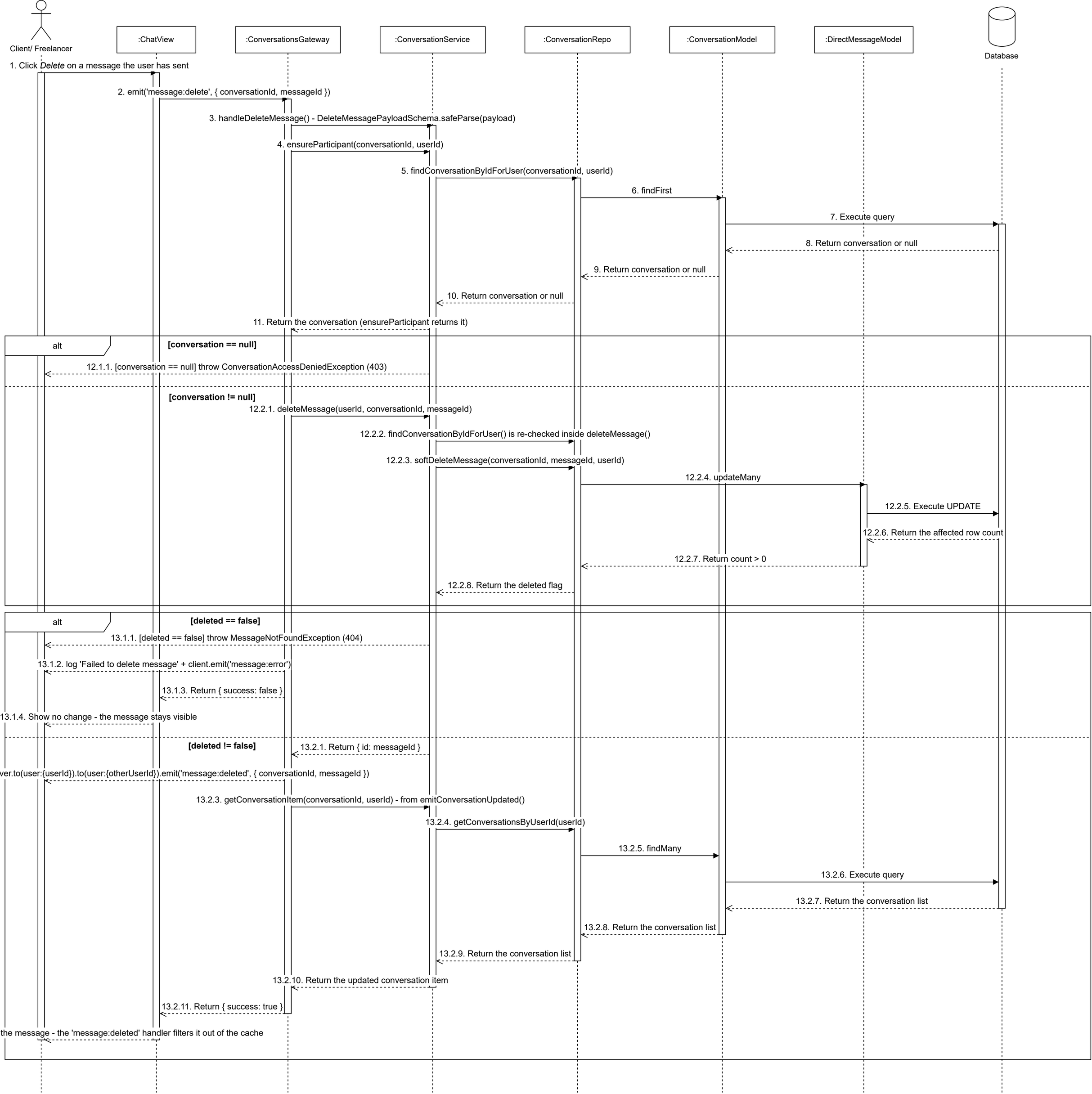

Client/ Freelancer
:ChatView
:ConversationsGateway
:ConversationService
:ConversationRepo
:ConversationModel
:DirectMessageModel
Database
1. Click *Delete* on a message the user has sent
2. emit('message:delete', { conversationId, messageId })
3. handleDeleteMessage() - DeleteMessagePayloadSchema.safeParse(payload)
4. ensureParticipant(conversationId, userId)
5. findConversationByIdForUser(conversationId, userId)
6. findFirst
7. Execute query
8. Return conversation or null
9. Return conversation or null
10. Return conversation or null
11. Return the conversation (ensureParticipant returns it)
alt
[conversation == null]
12.1.1. [conversation == null] throw ConversationAccessDeniedException (403)
[conversation != null]
12.2.1. deleteMessage(userId, conversationId, messageId)
12.2.2. findConversationByIdForUser() is re-checked inside deleteMessage()
12.2.3. softDeleteMessage(conversationId, messageId, userId)
12.2.4. updateMany
12.2.5. Execute UPDATE
12.2.6. Return the affected row count
12.2.7. Return count > 0
12.2.8. Return the deleted flag
alt
[deleted == false]
13.1.1. [deleted == false] throw MessageNotFoundException (404)
13.1.2. log 'Failed to delete message' + client.emit('message:error')
13.1.3. Return { success: false }
13.1.4. Show no change - the message stays visible
[deleted != false]
13.2.1. Return { id: messageId }
13.2.2. server.to(user:{userId}).to(user:{otherUserId}).emit('message:deleted', { conversationId, messageId })
13.2.3. getConversationItem(conversationId, userId) - from emitConversationUpdated()
13.2.4. getConversationsByUserId(userId)
13.2.5. findMany
13.2.6. Execute query
13.2.7. Return the conversation list
13.2.8. Return the conversation list
13.2.9. Return the conversation list
13.2.10. Return the updated conversation item
13.2.11. Return { success: true }
13.2.12. Remove the message - the 'message:deleted' handler filters it out of the cache